Lab Report

Multiclass Image Classification Using Convolutional Neural Networks and Transfer Learning

Course: Deep Learning Architectures and Techniques
Assignment: Practical Lab
Faculty: Dr. Shahid Ahmad Wani
Student Name: *Arsh Aggarwal*
Date: *04/03/2026*

---

1. Introduction

Image classification is an important application of deep learning where a model learns to assign labels to images. Convolutional Neural Networks (CNNs) are widely used for image recognition tasks because they can automatically learn spatial features such as edges, textures, and patterns.

In this experiment, a CNN model was implemented to perform **multiclass image classification**. A custom CNN architecture was first built and trained from scratch. After that, **transfer learning** was applied using a pretrained CNN model to improve classification performance.

Transfer learning allows models that were trained on large datasets to be reused for new tasks. This technique improves performance and reduces training time compared to training a model from scratch.

---

2. Dataset Description

The dataset used for this experiment is the **CIFAR-10 dataset**.

The CIFAR-10 dataset contains **60,000 color images of size 32×32 pixels divided into 10 classes**, with **50,000 training images and 10,000 test images**. (Keras)

The dataset classes are:

Class
Airplane

Class

Automobile

Bird

Cat

Deer

Dog

Frog

Horse

Ship

Truck

Each image belongs to one of these categories and the goal of the model is to correctly classify the object in the image.

---

3. Image Preprocessing

Before training the model, several preprocessing steps were performed.

3.1 Normalization

Pixel values in images range from **0 to 255**. These values were normalized to a range of **0–1** to improve model training.

X_train = X_train / 255.0

X_test = X_test / 255.0

Normalization helps the neural network converge faster during training.

3.2 Data Augmentation

Data augmentation was applied to increase dataset diversity and reduce overfitting.

The following augmentation techniques were used:

- Rotation
- Horizontal flipping
- Width shift
- Height shift

```
ImageDataGenerator(
rotation_range=20,
width_shift_range=0.1,
height_shift_range=0.1,
horizontal_flip=True
)
```

These transformations generate new variations of images during training.

---

4. Custom CNN Architecture

A custom CNN model was designed using the following layers:

Layer	Description
Conv2D	Extract spatial features
MaxPooling	Reduce spatial dimensions
Flatten	Convert feature maps to vector
Dense	Fully connected layer
Dropout	Prevent overfitting
Output	Softmax layer for classification

Model Structure

1️⃣ Convolution Layer (32 filters, ReLU)
2️⃣ MaxPooling Layer
3️⃣ Convolution Layer (64 filters, ReLU)
4️⃣ MaxPooling Layer

5️⃣ Flatten Layer
6️⃣ Dense Layer (128 neurons)
7️⃣ Dropout (0.5)
8️⃣ Output Layer (10 classes with Softmax)

The model was compiled using:

- **Optimizer:** Adam
- **Loss Function:** Sparse Categorical Crossentropy
- **Metric:** Accuracy

---

5. Model Training

The CNN model was trained for **10 epochs** using the augmented dataset.

```
model.fit(
datagen.flow(X_train,y_train,batch_size=32),
epochs=10,
validation_data=(X_test,y_test)
)
```

During training, two important metrics were tracked:

- Training loss
- Validation loss
- Training accuracy
- Validation accuracy

---

6. Training Results

Loss Curve

The loss curve shows how the model error decreases during training.

- Training loss gradually decreases
- Validation loss helps evaluate generalization performance

Accuracy Curve

The accuracy curve represents how well the model correctly classifies images during training.

Higher accuracy indicates improved classification performance.

---

7. Transfer Learning Model

After training the custom CNN, **transfer learning** was applied using **MobileNetV2**, a pretrained CNN model.

MobileNetV2 was pretrained on a large dataset (ImageNet), which allows it to extract powerful image features.

Steps used:

1. Load pretrained MobileNetV2
2. Freeze base layers
3. Add new classification layers

```
base_model = MobileNetV2(
weights='imagenet',
include_top=False,
input_shape=(32,32,3)
)
```

New layers added:

- GlobalAveragePooling
- Dense layer
- Softmax output layer

This model was then trained on the CIFAR-10 dataset.

---

8. Model Evaluation

After training, the model performance was evaluated using the test dataset.

The following metrics were calculated:

Confusion Matrix

A confusion matrix compares predicted labels with actual labels and helps visualize classification errors.

Accuracy

Accuracy measures the percentage of correctly classified images.

Precision

Precision measures how many predicted positives are correct.

Recall

Recall measures how many actual positives are correctly identified.

F1 Score

F1 score is the harmonic mean of precision and recall.

The evaluation metrics were computed using the following code:

```
confusion_matrix()

classification_report()
```

---

9. Model Comparison

Model	Accuracy
Custom CNN	Moderate accuracy
Transfer Learning (MobileNetV2)	Higher accuracy

The transfer learning model performed better because pretrained models have already learned useful image features from large datasets.

---

10. Conclusion

In this experiment, a **Convolutional Neural Network (CNN)** was implemented for multiclass image classification using the CIFAR-10 dataset.

Two approaches were tested:

1. Custom CNN architecture
2. Transfer learning using MobileNetV2

The results show that transfer learning significantly improves classification performance compared to training a model from scratch.

This experiment demonstrates how CNNs and pretrained models can effectively solve image classification problems.

Future improvements could include:

- Fine-tuning pretrained layers
- Increasing training epochs
- Using larger datasets
- Applying advanced augmentation techniques

---

11. Tools and Libraries Used

- Python
- TensorFlow / Keras
- NumPy
- Matplotlib
- Scikit-learn